Brightwheel AI Onboarding Triage – Sachin Veeramalla

What I Built and Why:
At 200 messages a week and 3 to 5 minutes per message, manual triage consumes up to 17 hours of specialist time weekly. That is roughly half a full-time role doing work that produces no customer outcome. Every hour spent triaging is an hour not spent helping a school get live on the platform. This system eliminates that bottleneck.

I built a web-based AI triage system that automates the entire triage workflow. A specialist pastes a message into a form, hits Submit, and within 10 seconds receives a structured output: message category, urgency level, which team owns it, and a draft reply they can edit and send directly from the tool. The system also accepts bulk CSV uploads so an entire week of messages can be processed in one batch with results organized by priority.

I evaluated three approaches. A Python script has no interface for non-technical users and requires a local machine to demo. An self-hosted n8n workflow requires a Docker container running before anything works, creates CORS issues when building a custom UI on top of webhook endpoints, and depends on a local machine staying available throughout the demo. A Next.js web application deployed on Vercel has none of these constraints. It works from any machine, has no local dependencies, and handles the interface and API in a single codebase. That is what I built.

The system uses an orchestrated pipeline of five specialized AI agents. Rather than asking one model to classify, prioritize, route, and draft a reply all at once, each agent does one job and passes its output to the next. This mirrors the four decisions a specialist makes manually and keeps each step independently debuggable. The 25-message CSV is stored as a reference dataset. For every new message, the three most relevant examples are retrieved and shown to the classifier as context, grounding decisions in real onboarding patterns.

Key Design Decisions and Tradeoffs:
1. Five agents instead of one. When one model handles everything, quality degrades on ambiguous messages and you cannot tell which decision went wrong. With five focused agents, if routing produces wrong results you fix the routing agent without touching anything else. The tradeoff is speed. Five sequential calls takes around 10 seconds versus 2 to 3 seconds for a single call. For 200 messages a week this is acceptable. Reliability matters more than speed here.

2.Keeping specialists in control of every outgoing reply. The system could send replies automatically. It deliberately does not. Every draft requires the specialist to review and click Send. An automated reply sent to a school opening tomorrow that gets something wrong causes more damage than the few seconds of review time. The system eliminates the work before that point while keeping accountability for what reaches the customer.

3. Privacy incidents bypass the pipeline entirely. A parent seeing another family's child's data is not an onboarding support ticket. It is a potential COPPA and FERPA compliance incident. The moment the classifier identifies a privacy incident, the pipeline short-circuits, the message routes to legal compliance, and a fixed response tells the specialist not to reply through standard channels. No draft reply is ever generated. Over-escalating is the right failure mode here.

Where This Breaks
Vague messages with no actionable detail cannot be meaningfully triaged and the clarifying question the system generates may not be the right one. Multi-topic messages produce one output rather than splitting into separate tickets. Draft replies are appropriate in tone but generic since the system has never seen Brightwheel's help documentation or product flows. Most critically, when the system produces a confident but wrong result and a specialist trusts it without checking, the error reaches the customer. Human review flags help but do not eliminate this risk entirely.

What Is Needed for Production

Brightwheel processes data on children under 13, which means COPPA applies directly. Sending raw message content to an external AI API is a compliance risk that needs legal sign-off before any real customer data touches the system. The practical fix is redacting personally identifiable information before the API call using a tool like Microsoft Presidio and establishing a data processing agreement with the AI provider.

Beyond privacy, the classification taxonomy was built from 25 messages and the system has no visibility into how Brightwheel actually responds when something goes wrong. Without knowing whether a P1 triggers a Slack alert, a phone call, or a ticket in an existing system like Zendesk, the automation cannot close the loop on its own most critical outputs. That operational context needs to come from the team before anything goes live.

What I Would Build Next

The most impactful next step is connecting the triage output to the onboarding team's existing CRM or ticketing system. Right now a specialist gets the result and still has to manually create a task in whatever platform the team uses to track work. That handoff is the remaining manual step. If a triaged message automatically creates an assigned ticket with the correct priority and owner, the time from message received to someone working on it drops to near zero. After that, connecting the reply agent to Brightwheel's help documentation would improve reply quality from appropriate to genuinely useful, and the correct and incorrect ratings already being collected would drive continuous improvement in classification accuracy over time.